Università degli Studi di Messina
Dipartimento di Scienze Matematiche e Informatiche,
Scienze Fisiche e Scienze della Terra (MIFT)
Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

PROGETTO 1:

Server HTTP Concorrente per la Prenotazione di Aule Studio

Massimo Mantineo (Matricola: 541924)

Pierluca Tino Castorina (Matricola: 545101)

Messina, Anno Accademico 2024/2025

Indice

1 Introduzione **2**
1.1 Obiettivi del Progetto 2

2 Architettura del Sistema **3**
2.1 Componenti dell'Architettura 3

3 Sincronizzazione e Gestione della Memoria **4**
3.1 Pattern Produttore-Consumatore con Semafori 4
3.2 Mutua Esclusione sui Dati delle Prenotazioni 4

4 Livello Applicativo: Parsing HTTP ed API REST **6**
4.1 Basic Authentication 6
4.2 Interfaccia CRUD (REST API) 6

5 Risultati Sperimentali: Load Testing **7**
5.1 Performance Metrics 7

6 Conclusioni e Sviluppi Futuri **8**
6.1 Potenziali Ottimizzazioni 8

1 Introduzione

Il presente documento descrive la progettazione e lo sviluppo di un server HTTP concorrente in linguaggio C, realizzato come progetto per il corso di Laboratorio di Reti e Sistemi Distribuiti. L'applicativo è stato concepito per fornire un servizio di backend dedicato alla gestione delle prenotazioni di aule studio universitarie.

A differenza dell'utilizzo di framework ad alto livello, il progetto mira a dimostrare la padronanza delle API POSIX di basso livello per la gestione dei socket di rete, la concorrenza tramite thread e la sincronizzazione dell'accesso a risorse condivise. Il server implementa il protocollo HTTP/1.1 per l'elaborazione delle richieste RESTful e utilizza un meccanismo di *Basic Authentication* per garantire la sicurezza degli endpoint sensibili.

1.1 Obiettivi del Progetto

Le specifiche e gli obiettivi tecnici raggiunti includono:

- **Gestione Concorrente**: Implementazione di un'architettura basata su *Thread Pool* per ottimizzare le prestazioni e ridurre l'overhead associato alla creazione dinamica di processi o thread per singola connessione.
- **Sincronizzazione e Sicurezza della Memoria**: Risoluzione di problemi di *race condition* mediante l'impiego di semafori (per la coda dei task) e mutex (per l'accesso esclusivo al gestore delle prenotazioni in memoria).
- **Protocollo HTTP**: Parsing manuale dei pacchetti HTTP, estrazione degli header e del payload, con particolare attenzione alla gestione delle richieste HTTP `GET`, `POST`, `PUT` e `DELETE`.
- **Validazione delle Prestazioni**: Esecuzione di *load testing* al fine di misurare il *throughput* massimo e verificare la stabilità del sistema sotto carico.

2 Architettura del Sistema

Per soddisfare il requisito della gestione simultanea di client multipli limitando al contempo il consumo di risorse hardware, si è optato per il **Pattern a Thread Pool**. Questo approccio pre-alloca all'avvio del server un numero fisso di thread lavoratori (*Worker Threads*) che permangono in attesa di richieste, limitando i cambi di contesto (context-switching) del kernel rispetto all'approccio classico con `fork()` *per-connection*.

2.1 Componenti dell'Architettura

Il sistema è logicamente suddiviso in tre strati principali:

1. **Main Thread (Dispatcher)**: Si occupa esclusivamente di ascoltare sulla porta TCP 8080 tramite la *system call* `accept()`. Una volta stabilita una connessione, il file descriptor risultante viene incapsulato in un task e inserito all'interno della coda di lavoro.

2. **Job Queue (Coda dei Task)**: Una coda FIFO (First-In, First-Out) implementata come lista concatenata e protetta da primitive di sincronizzazione.

3. **Worker Threads (Elaboratori)**: I thread estraggono i task dalla coda, effettuano il parsing della richiesta HTTP, interagiscono con il Resource Manager ed inviano la risposta al client.

4. **Resource Manager**: Un gestore centralizzato che incapsula la memoria delle prenotazioni (struttura dati) ed assicura le proprietà ACID minime sfruttando *lock* esclusivi.

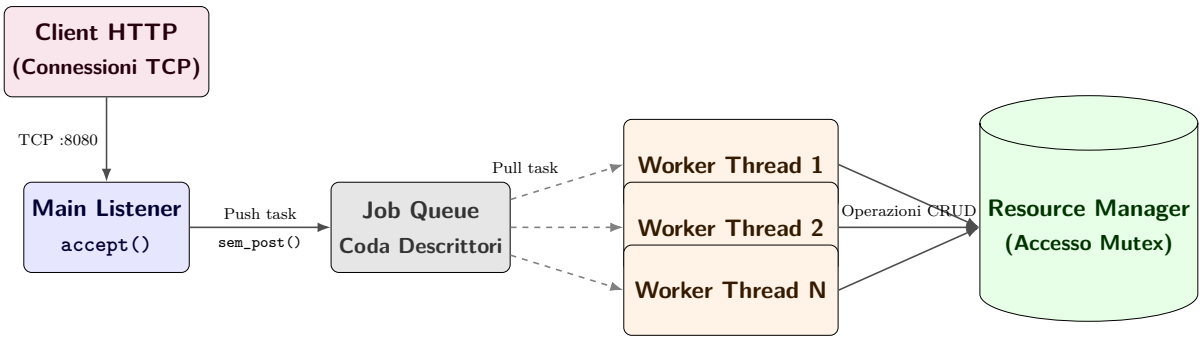

Figure 1: Architettura a Thread Pool e Sincronizzazione.

3 Sincronizzazione e Gestione della Memoria

Il problema principale riscontrato nei server concorrenti è legato alla manipolazione parallela di risorse condivise (in questo caso, la memoria interna in cui sono salvate le prenotazioni e la coda dei task). Per risolvere questi problemi, sono state impiegate due tipologie di lock forniti dalla libreria `pthread`: Mutex e Semafori.

3.1 Pattern Produttore-Consumatore con Semafori

Per gestire l'inserimento e l'estrazione dalla `Job Queue`, è stato applicato il classico paradigma *Producer-Consumer*.

- Il dispatcher agisce da **Produttore**. Creato un job, incrementa il conteggio del semaforo con `sem_post(&pool->queue_sem)`.
- I *Worker* sono i **Consumatori**. All'avvio chiamano `sem_wait(&pool->queue_sem)`. A coda vuota, il kernel blocca il thread in stato di *sleep*, evitando di consumare cicli di CPU (*busy-waiting*).

3.2 Mutua Esclusione sui Dati delle Prenotazioni

Mentre il dispatcher e i worker comunicano tramite semafori per la distribuzione del carico di rete, l'applicativo utilizza un **Mutex** (*Mutual Exclusion*) all'interno del `Resource Manager` per prevenire letture e scritture parziali e le corruzioni della memoria durante il parallelismo.

Esempio di sincronizzazione CRUD

```c
int rm_create(resource_manager_t* rm, const char* room, ...) {
    // 1) Acquisizione del lock in via esclusiva
    pthread_mutex_lock(&rm->mutex);

    if (rm->count >= MAX_RESERVATIONS) {
        pthread_mutex_unlock(&rm->mutex);
        return -1;
    }

    // 2) Operazione di scrittura critica
    strncpy(rm->reservations[rm->count].room, room, ...);
    rm->count++;
    int assigned_id = rm->reservations[rm->count - 1].id;

    // 3) Rilascio del lock per consentire l'accesso agli altri Worker
    pthread_mutex_unlock(&rm->mutex);
    return assigned_id;
}
```

Questo approccio granulare garantisce **Thread Safety** al 100% per l'array interno.

4 Livello Applicativo: Parsing HTTP ed API REST

Le stringhe inviate tramite socket non sono formattate a monte, consistono semplicemente in un buffer sequenziale di byte. Il file `http_parser.c` funge da *scanner* lessicale per decodificare le direttive del client analizzando i ritorni a capo standard di Windows/DOS (`\r\n`).

4.1 Basic Authentication

Per le rotte sensibili del sistema (es. `POST`, `PUT`, `DELETE`), è richiesto l'invio dell'header `Authorization`. Il server applica un sistema di validazione basato su crittografia Base64, confrontando nativamente la stringa decodificata per accertarsi che le credenziali (es. `admin:password`) corrispondano al token memorizzato prima di permettere il proseguimento dell'esecuzione, mitigando i tentativi d'intrusione.

4.2 Interfaccia CRUD (REST API)

L'interazione con il server avviene seguendo i principi architetturali REST. I dati ritornati dal sistema sono formattati in JSON, costrutto compatibile con i principali linguaggi frontend moderni.

La seguente tabella mappa le direttive HTTP sull'array in memoria:

Metodo HTTP	**Endpoint**	**Operazione Interna (C)**	**Status Code**
`GET`	`/reservations`	Mutex Lock → Lettura array JSON → Mutex Unlock	`200 OK`
`POST`	`/reservations`	Parsing URL-Encoded. Ricerca slot libero, scrittura struct e incremento counter.	`201 Created`
`PUT`	`/reservations/{id}`	Ricerca lineare dell'ID e aggiornamento dei campi.	`200 OK`
`DELETE`	`/reservations/{id}`	Rimozione elemento e shift a sinistra per ricompattare l'array.	`204 No Content`

Table 1: Mapping delle operazioni REST alle logiche di sistema.

5 Risultati Sperimentali: Load Testing

Per certificare il comportamento dei *Mutex* e misurare il throughput del server, è stata eseguita una simulazione locale tramite `Apache Bench (ab)`.

L'obiettivo dello Stress Test è di esaurire e mettere sotto carico l'applicativo eseguendo **10.000 richieste simultanee**, simulando un massimo di 100 utenti concorrenti paralleli che tentano l'interazione REST `GET` e il parsing crittografico su token Base64 in Authorization.

Avvio del Test prestazionale

```
$ ./bin/server_http &
$ ab -n 10000 -c 100 -H "Authorization: Basic
  YWRtaW46cGFzc3dvcmQ=" \
  http://localhost:8080/reservations
```

5.1 Performance Metrics

I risultati empirici registrati confermano la stabilità ingegneristica dell'infrastruttura:

- **Richieste Gestite:** 10.000 su 10.000 (Tasso di Successo del 100%).
- **Timeout / Errori di Race Condition:** Nessuno (0 fallimenti), a dimostrazione che il *Mutex* ha prevenuto efficacemente il collasso di memoria (assenza di *Segmentation Fault*).
- **Throughput Misurato:** $\sim$ 33.229 *Requests Per Second* (Req/Sec).
- **Latenza Mediana:** Il tempo medio di connessione, parsing ed erogazione si è assestato intorno ai 3.0 millisecondi, valore che suggerisce l'assenza di ingorghi all'interno dello *scheduler* POSIX.

6 Conclusioni e Sviluppi Futuri

Il progetto analizzato ha risposto pienamente ai requisiti forniti in ambito accademico, implementando uno stack protocollare robusto senza l'utilizzo di dipendenze esterne. Il passaggio dal paradigma per connessione `fork()`-based al modello efficiente a *Thread Pool* ha consentito all'applicativo di sostenere importanti carichi applicativi mantenendo costantemente basso l'overhead della memoria ed eliminando in modo netto i fallimenti strutturali.

6.1 Potenziali Ottimizzazioni

A fronte di uno scenario *Enterprise*, l'architettura si presta a ulteriori implementazioni avanzate:

1. **Database Persistente**: Sostituire l'uso esclusivo della RAM e dei *Mutex* logici con uno strato di persistenza vero e proprio (ad esempio agganciando driver nativi per `SQLite3` o `PostgreSQL`) per prevenire la perdita dei dati alla chiusura del server.

2. **Event-Loop I/O (`epoll`)**: Invece delle chiamate bloccanti `read()`, l'uso di `epoll` renderebbe i socket asincroni, impedendo che un worker thread resti bloccato da un client lento.

3. **Sicurezza TLS**: L'aggiunta dello strato OpenSSL garantirebbe il protocollo HTTPS impedendo l'intercettazione testuale in chiaro ed il furto degli Header HTTP.